Khadeja Qureshi - 27200

Lab 7 – GPGPU Programming

1) What are the two main attributes of computations that map well to GPUs?

The article says the two key attributes are data parallelism and independence. In other words, the same computation is applied to many data elements, and each element can usually be processed with little or no dependence on the others.

2) Explain the concept of "arithmetic intensity." Why is it important for GPU performance?

Arithmetic intensity is the ratio of computation to communication:
arithmetic intensity = operations / words transferred

It matters because GPUs are built to deliver very high computational throughput, but memory communication is relatively more expensive. So, workloads that do a lot of computation for each word of data moved tend to perform best on GPUs.

3) Why are grid-based simulations (like the cloud simulation example) useful when understanding GPU computing concepts?

Grid-based simulations are useful because they map naturally onto GPU data structures: grids map well to textures. They also match the GPU's strength in processing large regular collections of data in parallel, and GPUs have texture caches optimized for 2D locality, which helps this style of computation. Cloud simulation is a good teaching example because each step updates the whole grid, making it easy to think in terms of streams and kernels.

4) In the context of GPU computing, what is the difference between gather and scatter communication?

- **Gather**: the kernel processing one element reads information from other elements.
- **Scatter**: the kernel processing one element writes/distributes information to other elements.

The article also notes that gather only needs random-access reads, while scatters need random-access writes.

5) Name and describe the two types of programmable processors on a GPU discussed in the article.

The two programmable GPU processors discussed are:

Vertex processors
They process streams of vertices and run vertex programs/shaders that transform vertex data such as position, color, and normals. Because they can affect where pixels are eventually written, they are associated with scatter-like capability.

Fragment processors
They process streams of fragments (proto pixels) and run fragment programs/shaders to compute

final pixel values. They can fetch from textures, so they support gather well, and they are the main workhorse for many GPGPU tasks.

6) What is one major limitation of fragment processors when it comes to general-purpose computation?

A major limitation is that fragment processors cannot natively scatter. Their output location is fixed before processing, so they cannot choose arbitrary output addresses while running.

7) What is the GPU analogy for a CPU array and loop when programming a data-parallel computation?

The article gives these analogies:

- CPU array => GPU texture
- CPU inner loop =>GPU fragment program

So instead of writing a loop that iterates over every element of an array one at a time, you write a fragment program that expresses the same logic, and the GPU applies it to all elements in parallel across its many processors at once.

8) Why is render-to-texture important for feedback in GPGPU programming?

Most real computations have multiple steps, and later steps depend on the results of earlier ones. On a CPU this is trivial because you just read and write the same memory. On a GPU, though, memory reads and writes are separate. Render-to-texture bridges this gap: it lets the GPU write the output of one pass directly into texture memory, which can then be fed as input into the next pass, all without ever sending data back to the CPU. Without it, implementing any kind of iterative or multi-step algorithm on the GPU would be extremely painful.

9) What is a reduction operation in GPU computing, and why is it more challenging to implement efficiently on a GPU compared to a CPU?

A reduction combines many values into fewer values, often down to a single result, such as a sum or maximum. On the GPU, this is harder because GPUs are optimized for computations where many elements are processed independently. Reductions require multiple passes, repeatedly rendering and reading buffers while shrinking the output each step, which takes about O(log n) passes. That multi-pass structure makes it less straightforward than a CPU implementation.

10) According to Table 31-1, what are the main differences in floating-point formats supported by NVIDIA and ATI GPUs, and how do these differences affect computation precision?

The table reveals some meaningful hardware differences between the two manufacturers at the time:

- NVIDIA supported both a 16-bit and a 32-bit (IEEE 754) format. The 32-bit format with 23 mantissa bits and 8 exponent bits offered the best precision of anything in the table, with exact integer representation up to +/-16,777,216 and support for special values like NaN and infinity.

- ATI supported a 24-bit format (16 mantissa bits, 7 exponent bits) instead of a full 32-bit. It landed between the two NVIDIA formats in terms of precision, with a whole-number range of +/-131,072, but critically, ATI's formats don't support NaN or infinity at all.

In practice, this meant NVIDIA's 32-bit mode was the safest choice for numerically sensitive work, while ATI's lack of special-value support could cause silent, hard-to-debug errors in edge cases. The 16-bit formats on both sides traded precision for speed, with the smallest representable whole-number range (+/-2048).